

MANIPAL UNIVERSITY JAIPUR
Centre for Distance and Online Education

INTERNAL ASSIGNMENT

Course Code & Name: DCA2108 – Operating Systems

Program: Bachelor of Computer Applications (BCA)

Semester: III

Session: September 2025

Name: Hirdyansh Varshney

Roll No.: 2414500172

Course: BCA

Subject: Operating Systems

Assignment: Set-I & Set-II

Submission Date: 01-11-2025

Submitted To:
Manipal University Jaipur
Centre for Distance and Online Education

Set – 1

Question 1:

Answer:

What is a PCB?

A Process Control Block (PCB) is a vital data structure used by the operating system to manage and track processes. Every time a new process is created, the OS generates a corresponding PCB to store essential details about that process. It acts like an identity card for the process, containing all the information the OS needs to control and monitor its execution.

The PCB enables multitasking by allowing the OS to switch between processes efficiently. When a process is paused or interrupted, its current state is saved in the PCB. Later, the OS can resume the process using the stored data. This mechanism is crucial for implementing features like context switching, scheduling, and resource allocation.

PCBs are stored in a protected area of memory to prevent unauthorized access. They are linked together in queues, such as ready, waiting, or terminated, depending on the process state. The OS uses these queues to decide which process to run next.

In summary, the PCB is the backbone of process management in modern operating systems. Without it, the OS would struggle to maintain control over multiple processes, leading to inefficiency and instability.

What all information is stored in a PCB

A PCB holds a wide range of information that helps the operating system manage a process throughout its lifecycle. The most critical data stored in a PCB includes:

- **Process ID (PID):** A unique identifier assigned to each process.
- **Process State:** Indicates whether the process is running, ready, waiting, or terminated.
- **Program Counter:** Stores the address of the next instruction to be executed.
- **CPU Registers:** Contains the values of all registers used by the process.
- **Memory Management Info:** Includes pointers to memory segments like code, data, and stack.
- **Scheduling Information:** Priority level, scheduling queue pointers, and other scheduling parameters.
- **I/O Status:** Tracks the list of I/O devices allocated to the process and pending I/O requests.
- **Accounting Info:** Includes CPU usage, execution time, and other performance metrics.
- **File Descriptors:** References to files opened by the process.
- **Security and Privileges:** User ID, group ID, and access control data.

This comprehensive set of information allows the OS to pause, resume, or terminate processes accurately. It also ensures that resources are allocated fairly and that system stability is maintained during multitasking operations.

Question 2:

Answer:

What is Inter-Process Communication (IPC)

Inter-Process Communication (IPC) refers to the mechanisms that allow processes to exchange data and coordinate their actions within an operating system. Since modern systems often run multiple processes simultaneously, IPC becomes essential for enabling these processes to work together effectively.

Processes are typically isolated from one another to maintain security and stability. IPC bridges this isolation by providing controlled pathways for communication. These pathways can be implemented using various methods such as pipes, message queues, shared memory, semaphores, and sockets. Each method has its own advantages depending on the nature of the communication—whether it's synchronous, asynchronous, or requires data sharing.

For example, in a client-server model, the server process may use sockets to receive requests from multiple client processes. Similarly, shared memory allows two processes to access a common memory space, making data exchange faster but requiring synchronization to avoid conflicts.

IPC is a core concept in operating systems, especially in multitasking environments. It ensures that processes can collaborate without compromising system integrity. Whether it's coordinating tasks, sharing resources, or passing messages, IPC provides the foundation for process interaction.

Why IPC is Important

IPC plays a critical role in the overall functionality and efficiency of an operating system. Without IPC, processes would operate in isolation, unable to share information or coordinate actions. This would severely limit the system's ability to perform complex tasks or support multi-user environments.

One major reason IPC is important is resource sharing. For example, multiple processes may need access to a printer or a database. IPC allows them to communicate their needs and manage access without conflict. It also supports modular programming, where different components of an application run as separate processes and interact through IPC.

Another key benefit is synchronization. IPC mechanisms like semaphores and message queues help processes stay in sync, ensuring that operations occur in the correct order. This is vital in real-time systems where timing and coordination are crucial.

IPC also enhances performance by enabling parallel processing. Tasks can be divided among multiple processes that communicate and collaborate, leading to faster execution and better resource utilization.

In summary, IPC is essential for building responsive, efficient, and scalable systems. It allows processes to work together seamlessly, making modern computing possible.

Question 3:

Answer:

Differences between SJF and Round Robin Scheduling

Shortest Job First (SJF) and Round Robin (RR) are two widely used CPU scheduling algorithms, each with distinct characteristics and use cases.

SJF is a non-preemptive scheduling method where the process with the shortest burst time is executed first. It aims to minimize average waiting time and turnaround time. However, it requires prior knowledge of process durations, which is not always feasible. SJF can lead to starvation, where longer processes are delayed indefinitely if shorter ones keep arriving.

Round Robin, on the other hand, is a preemptive scheduling technique designed for time-sharing systems. Each process is assigned a fixed time slice (quantum), and the CPU cycles

through the ready queue, giving each process a fair chance to execute. If a process doesn't finish within its time slice, it's moved to the back of the queue. This ensures responsiveness and prevents starvation.

The key difference lies in fairness and predictability. SJF is optimal in terms of efficiency but can be unfair to longer tasks. RR is fair and simple but may increase average turnaround time due to frequent context switching.

Example of SJF and Round Robin Scheduling

Consider three processes with burst times:

- P1: 4 ms
- P2: 2 ms
- P3: 6 ms

In SJF, the execution order would be $P2 \rightarrow P1 \rightarrow P3$, minimizing waiting time.

In Round Robin (quantum = 2 ms), the order would cycle: $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1 \rightarrow P3 \rightarrow P3$, ensuring equal CPU time distribution.

Set – 2

Question 4:

Answer:

Banker's Algorithm: A Deadlock Avoidance Strategy

Banker's Algorithm is a classic deadlock avoidance technique used in operating systems to ensure safe resource allocation among multiple processes. Developed by Edsger Dijkstra, the algorithm is named after the way a banker might allocate funds to clients while ensuring the bank never runs out of money.

The core idea of Banker's Algorithm is to simulate resource allocation before actually committing to it. It checks whether the system will remain in a safe state if a process is granted its requested resources. A safe state means that there exists a sequence of process execution where each process can complete without causing a deadlock.

To implement the algorithm, the system maintains several data structures:

- **Available:** Tracks the number of available instances of each resource.
- **Max:** Indicates the maximum demand of each process.
- **Allocation:** Shows the current allocation of resources to each process.
- **Need:** Represents the remaining resource needs of each process (calculated as $\text{Max} - \text{Allocation}$).

When a process requests resources, the algorithm checks:

1. If the request is less than or equal to its remaining need.
2. If the requested resources are available.
3. If granting the request keeps the system in a safe state.

If all conditions are met, the resources are allocated. Otherwise, the process must wait.

Example

Consider three processes and three resource types. If Process P1 requests resources, the algorithm evaluates whether fulfilling the request will still allow P2 and P3 to complete. If yes, the system is safe; if not, the request is denied temporarily.

Conclusion

Banker's Algorithm is effective in preventing deadlocks but requires prior knowledge of maximum resource demands, which may not always be practical. Despite its limitations, it remains a foundational concept in operating system design for safe resource management.

Question 5:

Answer:

Deadlock Avoidance Measures in Operating Systems

Deadlock avoidance is a proactive strategy used by operating systems to ensure that resource allocation does not lead to a deadlock situation. A deadlock occurs when a group of processes are stuck waiting for each other's resources indefinitely, causing the system to freeze or slow down.

To avoid deadlocks, the OS uses algorithms that analyze resource requests before granting them. One of the most well-known techniques is the Banker's Algorithm. It checks whether granting a resource request will leave the system in a safe state, meaning all processes can eventually complete. If the system would become unsafe, the request is denied or delayed.

Another method is to impose resource ordering. The OS assigns a global order to all resources and ensures that processes request resources in that order. This prevents circular wait conditions, which are one of the four necessary conditions for deadlock.

Additionally, the OS may limit the number of resources a process can request at once or require processes to declare their maximum resource needs in advance. These constraints help the system predict and prevent potential deadlocks.

In summary, deadlock avoidance focuses on careful planning and analysis before resource allocation. It ensures that the system remains stable and responsive, even under heavy multitasking.

Deadlock Recovery Measures in Operating Systems

When a deadlock occurs despite preventive measures, the operating system must take steps to recover and restore normal operation. Deadlock recovery involves detecting the deadlock and then choosing a strategy to break it.

The first step is **deadlock detection**, where the OS periodically checks for cycles in the resource allocation graph. If a cycle is found, it indicates that a deadlock has occurred. The system then selects one or more processes to terminate or roll back to resolve the issue.

One recovery method is **process termination**. The OS may choose to kill one or more processes involved in the deadlock. This frees up resources and allows other processes to continue. The decision can be based on priority, execution time, or resource usage.

Another approach is **resource preemption**, where the OS forcibly takes resources away from certain processes and reallocates them. This method is more complex and may lead to inconsistent states, so it requires careful handling.

Process rollback is also used in systems that support checkpointing. The OS rolls back a process to a previous safe state and restarts it, avoiding the deadlock without terminating the process.

In conclusion, deadlock recovery is a reactive measure that helps the system recover from a blocked state. While it may disrupt some processes, it ensures that the overall system remains functional and responsive.

Question 6:

Answer:

I/O Overhead in Demand Paging and Its Impact on System Performance

Demand paging is a memory management technique where pages are loaded into RAM only when they are needed. While this approach conserves memory and improves flexibility, it introduces significant I/O overhead that can affect system performance.

One major source of overhead is **page faults**. When a process accesses a page that is not in memory, the operating system must retrieve it from secondary storage, usually a hard disk or SSD. This disk access is much slower than RAM, causing delays in execution. Frequent page faults can lead to thrashing, where the system spends more time swapping pages than executing actual processes.

Another contributor is **disk latency**. Even with fast storage devices, reading and writing pages involves mechanical or electronic delays. These delays accumulate when multiple processes compete for memory, leading to bottlenecks.

Context switching also adds overhead. When a page fault occurs, the OS must pause the current process, handle the fault, and resume execution. This switch consumes CPU cycles and increases response time.

Buffering and caching inefficiencies can further degrade performance. If the system's cache is not optimized, repeated access to the same pages may still trigger disk reads, wasting I/O bandwidth.

To mitigate these issues, several strategies can be employed:

1. **Page replacement algorithms** like LRU (Least Recently Used) help retain frequently accessed pages in memory.
2. **Pre-paging** anticipates future page requests and loads them in advance.
3. **Increasing RAM** reduces the need for frequent paging.
4. **Using SSDs** instead of HDDs lowers disk latency.
5. **Optimizing application memory usage** reduces unnecessary page faults.

In conclusion, while demand paging is efficient in memory usage, its I/O overhead can slow down system performance. Careful tuning of memory policies and hardware upgrades can significantly reduce these impacts.